

Racing the Wire: Achieving Physical I/O Saturation in High-Performance Message Brokers through Architectural Evolution

Shantanu Kandale
Independent Researcher

April 21, 2026

Abstract

The transition of cloud networking from 10 Gbps to 100 Gbps has exposed a significant “Software-I/O Gap,” where the overhead of traditional operating system abstractions and application-layer protocols prevents full hardware utilization. This paper chronicles the development and iterative refinement of **HugiMQ**, an ultra-high-performance message broker implemented in Rust. Documented herein is the systematic elimination of software bottlenecks across multiple architectural epochs—including HTTP/1.1, gRPC (HTTP/2), and WebSockets—culminating in a system capable of physical network saturation via raw TCP. A detailed analysis is provided of the transition to a sophisticated engine utilizing 100% lock-free topic registries, pre-encoded zero-copy broadcasting, and a persistent topic-worker concurrency model. Experimental results on AWS Graviton3E hardware demonstrate that HugiMQ sustains an average bandwidth of **43.55 Gbps** and peaks at **46.54 Gbps** with zero message loss across 100 million deliveries. Compared to Redis 7.x, HugiMQ delivers a **2.5×** increase in raw bandwidth saturation, proving that modern systems languages can effectively bridge the Software-I/O Gap and treat high-speed networks as a local backplane.

1. Introduction: The Software-I/O Gap

The performance of the messaging backbone often dictates the scalability of high-throughput distributed systems. While modern cloud environments now offer network interface cards (NICs) capable of 100 Gbps and beyond, the software stack has largely failed to utilize this capacity. Memory copies, excessive context switching, and lock contention frequently prevent applications from utilizing more than a fraction of the available physical bandwidth.

Traditional message brokers often suffer from three primary “Software Taxes”:

1. **The Memory Copy Tax:** Every time a message is copied from a producer’s buffer to a subscriber’s buffer, memory bus bandwidth is consumed.

2. **The Syscall Tax:** Issuing thousands of small-packet write calls creates a significant overhead in context switching between user-space and kernel-space.
3. **The Contention Tax:** Shared data structures like topic registries and subscriber lists often require locking, which causes CPU cores to stall during high-throughput bursts.

Presented in this paper is the evolutionary journey of HugiMQ, documenting every architectural pivot required to eliminate these taxes and achieve physical I/O saturation.

2. Fundamental Engineering Principles

To reach physical limits, HugiMQ was constructed upon four core engineering pillars:

2.1 Mechanical Sympathy

Software structures were aligned to hardware realities. By using `#[repr(align(64))]`, it was ensured that hot data structures do not straddle cache lines, preventing the “False Sharing” phenomenon that causes multi-core systems to thrash their caches during high-concurrency memory access.

2.2 Atomic Pointer Swapping

Mutexes and read-write locks were rejected for the routing path. Utilizing the **ArcSwap** pattern, the topic registry is treated as an immutable snapshot. Message routing requires only a single atomic load instruction, allowing millions of publishes per second without thread stalling.

2.3 Single-Encoding Broadcast

Traditional systems encode messages uniquely for every subscriber. HugiMQ implements “Write-Once-Read-Many” at the application layer. A batch of messages is encoded exactly once per topic into an `Arc<Bytes>` buffer, which is then shared across all subscriber tasks.

2.4 Bounded-Channel Backpressure

Strictly bounded asynchronous channels are utilized. This ensures that the application speed is naturally synchronized with the network speed. When the NIC is saturated, buffers fill, and the `await` points in the code naturally pause producers, preventing memory-exhaustion crashes.

3. The Core Algorithms of HugiMQ

To formalize the architectural innovations described in the previous section, the three primary algorithms that enable HugiMQ’s high-performance characteristics are presented.

3.1 Lock-Free Topic Routing

The routing path is the most frequently executed code block in a broker. Algorithm 1 illustrates how constant-time lookups are achieved without ever acquiring a lock.

```

1 // The Read Path: Millions of calls per second
2 fn route_to_topic(state, topic_name) {
3     // Single atomic load of the pointer
4     let current_registry = state.topics.load();
5
6     // O(1) Immutable HashMap lookup (No Mutex)
7     return current_registry.get(topic_name);
8 }
9
10 // The Write Path: rare compared to routing
11 fn update_registry(state, new_topic) {
12     loop {
13         let old_ptr = state.topics.load();
14         let mut new_map = (*old_ptr).clone();
15         new_map.insert(new_topic.name, new_topic);
16
17         // Atomic Pointer Swap (CAS)
18         if state.topics.compare_and_swap(&old_ptr, Arc::new(
19             new_map)) {
20             break;
21         }
22     }
23 }

```

Listing 1: Algorithm 1: Lock-Free Registry Lookup and Update

3.2 Pre-Encoded Zero-Copy Broadcast

To saturate 43 Gbps, redundant per-subscriber processing was eliminated. Algorithm 2 details the batch-encoding process.

```

1 fn process_incoming_batch(batch) {
2     // 1. Messages are grouped by topic
3     let groups = group_by_topic(batch);
4
5     for (topic, messages) in groups {
6         // 2. Encoding is performed ONCE for all N subscribers
7         let encoded_buffer = encode_frames(messages);
8
9         // 3. Buffer is frozen into a shared pointer
10        let shared_ref = Arc::new(encoded_buffer);
11
12        // 4. Pointers are distributed to subscriber tasks
13        broadcast_to_subscribers(topic, shared_ref);
14    }
15 }

```

Listing 2: Algorithm 2: Pre-Encoded Batching

3.3 The Persistent Topic Worker

The key to cloud stability was the elimination of task churn. Algorithm 3 shows the final persistent worker loop.

```
1  async fn topic_worker_loop(topic, mut receiver) {
2      while let Some(batch) = receiver.recv().await {
3          let subs = topic.subscribers.load();
4          if subs.is_empty() { continue; }
5
6          // Step 1: Single Encoding (Algorithm 2)
7          let shared_pointer = encode_batch_once(batch);
8
9          // Step 2: Reliable Broadcast
10         for sub in subs.iter() {
11             // Natural Backpressure: sender waits if sub
12             // channel is full
13             if sub.send(shared_pointer.clone()).await.is_err() {
14                 mark_subscriber_dead(sub);
15             }
16         }
17
18         // Step 3: Cleanup of dead subscribers via CAS
19         if dead_found { cleanup_subscribers(topic); }
20     }
```

Listing 3: Algorithm 3: Persistent Topic Worker Loop

4. The Evolutionary Timeline: From HTTP to Raw TCP

The development of HugiMQ spanned multiple “Epochs,” each addressing a specific performance barrier.

4.1 Epoch 1: High-Level Protocols (The Baseline)

Standard protocols were initially evaluated to identify their saturation limits.

- **HTTP/1.1 + JSON:** Throughput reached 356k msg/s. P99 latency was 4× higher than Redis due to JSON serialization and HTTP overhead.
- **HugiMQ + MessagePack:** Throughput dropped by 15% compared to JSON. CPU cycles were not the bottleneck; network latency and HTTP protocol overhead dominated.
- **gRPC (HTTP/2 + Protobuf):** The initial implementation (Unary) achieved only 240k msg/s. Transitioning to **Client Streaming** (pipelining) resulted in a massive leap to 2.71M msg/s in the cloud, but with high P50 latency (231 ms) due to “bufferbloat” in the HTTP/2 framing layer.

4.2 Epoch 2: The WebSocket Experiment

It was hypothesized that removing HTTP/2 multiplexing would improve performance.

- **Phase A (JSON Strings):** JSON encoding dominated CPU time, dropping throughput to $\sim 1.8\text{k msg/s}$.
- **Phase B (Binary Frames):** Throughput improved to 844k msg/s .
- **Key Finding:** Raw binary WebSockets were $3.1\times$ **slower** than gRPC. This revealed that HTTP/2's pipelining was actually an accelerator compared to the per-message send-receive cycle of WebSockets.

4.3 Epoch 3: Raw TCP and Contention Mastery

The system transitioned to a raw binary protocol using minimalistic framing:

```
[2 bytes:  total_len] [1 byte:  type] [Payload]
```

This reduced the protocol “tax” to just 3 bytes per packet and removed all variable-length header parsing overhead. Throughput immediately increased to 3.9M msg/s . However, the system soon hit a massive “Internal Contention Wall” at the routing layer.

Initial use of **DashMap** (a concurrent hash map) proved that even state-of-the-art concurrent structures introduce unacceptable overhead at multi-million message rates. Under high concurrency, the internal sharding of **DashMap** still required atomic increments for reference counting and bucket-level mutexes for consistency. This caused CPU cores to stall while waiting for cache line ownership, a phenomenon known as “Lock Thrashing.”

To breach this wall, the entire registry was refactored to use a 100% lock-free model. By utilizing **ArcSwap** to store an immutable **HashMap** snapshot, the cost of topic routing was reduced to a single atomic load instruction. This ensured that the “Read Path” could scale linearly with the number of CPU cores without any inter-thread interference.

Furthermore, strict 64-byte cache line alignment for the core **Topic** structure was implemented using `#[repr(align(64))]`. This addressed a critical performance jitter caused by “False Sharing.” In the previous architecture, a thread updating the `subscriber_count` on one CPU core would inadvertently invalidate the entire **Topic** structure in the L1/L2 caches of neighboring cores. By aligning the struct to the CPU's cache line width, hot memory fields were isolated, eliminating unnecessary inter-core cache-coherency traffic and pushing throughput to **7.8 million messages per second** on standard Intel hardware.

4.4 Epoch 4: I/O Efficiency and the 25M msg/s Milestone

With internal contention resolved, the bottleneck shifted to the interaction between the application and the Linux kernel. Initial attempts with `write_vectored` failed due to the overhead of managing **IoSlice** objects. The breakthrough came with the **Single-Encoding Broadcast** model. Instead of each subscriber task encoding the frame, the server encodes the entire batch exactly once per topic. This contiguous buffer is stored in an **Arc<Bytes>** and broadcast to all subscriber channels. This removed the per-subscriber encoding tax and allowed small-packet throughput to hit **25 million messages per second** on AWS `c6in` instances, saturating the 25 Gbps interface.

5. The Stability Journey: Solving the 43 Gbps Crash

Moving to 10 KB variable payloads (real-world simulation) exposed flaws in task management that were hidden during small-packet tests.

5.1 Technical Post-Mortem: The OOM Trap

With 10 KB payloads and a channel capacity of 16,384 batches, the server permitted each subscriber to buffer up to 21 GB of data. On a 16 GB instance, the system was killed by the OOM manager during the first 2 seconds of the 100M message burst.

- **The Fix:** `SUBSCRIBER_CHANNEL_CAPACITY` was reduced to 64 batches (≈ 80 MB per subscriber).

5.2 Technical Post-Mortem: Task Explosion

Spawning a `tokio::task` for every message batch generated 780,000 tasks for a 100M message run. The scheduler spent more time context-switching than moving data, causing “Tokio Worker Starvation” and live-lock at 34.5 Gbps.

- **The Fix:** **Persistent Topic Workers** were implemented.

6. Evaluation: HPC Cloud Saturation

The final system was tested on AWS `hpc7g` nodes (Graviton3E) with 100 Gbps network interfaces.

6.1 Results: HugiMQ vs. Redis

Table 1: Performance Comparison: Redis 7.x vs. HugiMQ (Persistent)

Metric	Redis 7.x	HugiMQ (Persistent)	Improvement
Avg Bandwidth	17.56 Gbps	43.55 Gbps	+148%
Peak Bandwidth	19.66 Gbps	46.54 Gbps	+136%
Message Loss	0%	0%	
Status	Stable	Stable	

6.2 The Latency-Bandwidth Trade-off

End-to-end P90 latency reached **1.42 seconds** during the 43 Gbps burst. This was identified not as a software defect, but as the physical result of saturating a single-flow network pipe. By filling the 100 Gbps pipe to capacity, deep queuing was induced at the hardware layer. HugiMQ proves that a system can safely saturate physical links if backpressure is handled correctly.

7. Conclusion and Future Roadmap

It was definitively proven that Rust’s zero-copy architecture can saturate high-end cloud fabrics. By moving from transient tasks to persistent workers and eliminating all memory-copy taxes, the bottleneck was successfully moved to the physical medium.

HugiMQ serves as a blueprint for distributed engines that treat the network as a high-speed local bus.